xAI - TimeLDM:
Latent Diffusion Model for Unconditional Time Series Generation

Iliass Khoutaibi
iliass.khoutaibi@telecom-paris.fr

Théo Basséras
theobasseras@gmail.com

Contents

1 Introduction **1**

2 TimeLDM: Time Latent Diffusion Model **1**

2.1 Variational Auto-encoding Architecture . . . 1

2.2 Diffusion Model . . . 2

2.3 Training . . . 3

2.4 Sampling . . . 3

3 Evaluation Metrics **3**

3.1 Qualitative Metrics . . . 4

3.2 Quantitative Metrics . . . 4

4 Experiments and Dataset **4**

5 Implementations **4**

5.1 Hyperparameters . . . 4

5.2 Implementation 1 . . . 5

5.3 Implementation 2 . . . 5

5.4 Results Analysis . . . 5

5.5 Quantitative Evaluation . . . 6

5.6 Discussion . . . 6

6 xAI analysis **6**

6.1 Definition of the target features . . . 7

6.2 Sobol' indices . . . 7

6.3 Series perturbation and channel ablation . . . 8

6.4 Counterfactual explanations . . . 8

7 Use of Artificial Intelligence **9**

7.1 Theoretical Clarification . . . 9

7.2 Technical Implementation and Debugging . . . 9

7.3 Methodological Exploration . . . 9

7.4 Editorial Assistance . . . 9

8 Conclusion and Future Work **10**

8.1 Conclusion . . . 10

8.2 Future Work . . . 10

9 Appendix **12**

9.1 The TimeLDM algorithm . . . 12

9.2 Different implementations results . . . 13

9.3 Highlighting Sobel "insensitivity" . . . 16

9.4 Series perturbation and channel ablation . . . 17

9.5 Counterfactual explanations . . . 17

1 Introduction

In this work, we investigate a latent diffusion model for time series generation, with a particular focus on understanding both its generative capabilities and its internal mechanisms. We begin by presenting the core ideas of the original paper, highlighting the motivation behind latent diffusion and its advantages over direct data-space generation. We then describe the architectures involved, including the Variational Autoencoder (VAE) used to encode time series into a latent space, and the diffusion model responsible for learning the generative process within this space.

We subsequently detail our implementation, discussing the design choices made for both the encoder-decoder structure and the denoising network, as well as the training procedure and sampling strategy. Finally, we introduce several explainability methods aimed at understanding how the model produces its outputs. In particular, we leverage global sensitivity analysis (Sobol' indices), local gradient-based saliency methods, and counterfactual reasoning. These complementary approaches allow us to analyze the contribution of latent variables, the role of the diffusion process, and the causal mechanisms underlying the generation of time series.

Our implementation can be found on this Github repositiory. (Make sure to star ★ it !)

2 TimeLDM: Time Latent Diffusion Model

In this section, we present the Time Latent Diffusion Model (TimeLDM), which forms the core of our generative framework. The model operates in a latent space learned by a Variational Autoencoder, where time series are encoded into a compact and structured representation. A diffusion process is then applied in this latent space, progressively transforming noise into meaningful latent trajectories through a learned denoising network. This approach enables high-quality generation while leveraging the expressiveness of latent representations and the stability of diffusion modeling.

Denote by $\mathbf{x}_{\tau:\mathbf{d}} = (\mathbf{x_1}, \cdots, \mathbf{x}_\tau)$ an input time series of sequence length τ with each timed element of dimension d. Given a time series data set $S = \{\mathbf{x}\}_{i=1}^N$, the goal of TimeLDM is to learn a parameterized generative model $p_\theta(S)$ which can generate a synthetic time series data set $\hat{S} = \{\hat{\mathbf{x}}\}_{i=1}^N$, with features that closely match the original.

2.1 Variational Auto-encoding Architecture

We first map the input time series $\mathbf{x}$ to a latent representation using a convolutional embedding layer, which captures local temporal patterns:

$$e = \text{Embedding}(\mathbf{x})$$

To preserve temporal ordering, we incorporate positional information by adding a learnable positional encoding p_e:

$$e_{pe} = e + p_e$$

The resulting representation combines local signal features with explicit temporal context, providing a structured input for subsequent processing.

A classical autoencoder learns a deterministic latent representation from which the input can be reconstructed with high fidelity. However, such latent spaces are often irregular and sensitive to perturbations, which limits their suitability for generative modeling.

To overcome this limitation, we adopt a variational formulation and introduce a **stochastic latent representation**. The embedded signal e_{pe} is processed by two **transformer** networks that parameterize the latent distribution:

$$\mu = T_1(e_{pe}), \qquad \sigma = T_2(e_{pe})$$

These networks estimate the mean and standard deviation of a Gaussian latent variable. A sample z is then obtained via the reparameterization trick:

$$z = \mu + \sigma\epsilon, \quad \text{with} \quad \epsilon \sim \mathcal{N}(0, I)$$

This formulation enforces a smooth and structured latent space, improving robustness to noise and enabling effective generative modeling.

The latent variable z is then decoded using a transformer-based architecture. The decoder leverages both self-attention and cross-attention mechanisms:

- **Self-attention** captures long-range dependencies and global structure within the generated time series.
- **Cross-attention** conditions the reconstruction on the latent representation, allowing the model to recover temporally coherent patterns.

This design enables the generation of realistic and structured time series directly from the latent space.

The autoencoder is trained using a combination of reconstruction and regularization losses.

The reconstruction objective $\mathcal{L}_{\text{recon}}$ enforces fidelity between the input time series $\mathbf{x}$ and its reconstruction $\hat{\mathbf{x}}$. It combines several complementary terms:

- an L_2 loss to capture global amplitude differences,
- an L_1 loss to improve robustness and preserve local variations,
- a Fourier-domain loss to ensure that periodic structures and frequency components are accurately reconstructed.

In addition, a Kullback–Leibler (KL) divergence term regularizes the latent space by encouraging the posterior distribution $q_\phi(\mathbf{z}|\mathbf{x})$ to match a Gaussian prior. This promotes a smooth and continuous latent representation, where similar time series are mapped to nearby regions, which is essential for stable sampling and generation.

The overall training objective is given by:

$$\mathcal{L}_{\text{VAE}} = \underbrace{\lambda_1 \|\mathbf{x} - \hat{\mathbf{x}}\|_2^2}_{\text{L2 loss}} + \underbrace{\lambda_2 \|\mathbf{x} - \hat{\mathbf{x}}\|_1}_{\text{L1 loss}} + \underbrace{\lambda_3 \|\mathcal{FFT}(\mathbf{x}) - \mathcal{FFT}(\hat{\mathbf{x}})\|_2}_{\text{FFT loss}} + \underbrace{\beta \cdot \text{KL}\left(q_\phi(\mathbf{z}|\mathbf{x}) \,\|\, \mathcal{N}(0, I)\right)}_{\text{KL divergence}}$$

2.2 Diffusion Model

Diffusion models have recently emerged as powerful generative models capable of learning complex data distributions through a gradual denoising process. In particular, denoising diffusion probabilistic models (DDPMs) [6] and score-based generative models [13] provide a principled framework based on stochastic processes. Further improvements and design insights have been proposed in [8].

In our framework, diffusion is performed in the latent space learned by the autoencoder. Let $\mathbf{z}_0$ denote the latent representation of a time series. The forward diffusion process progressively corrupts this latent variable by adding Gaussian noise over T timesteps:

$$q(\mathbf{z}_t \mid \mathbf{z}_0) = \mathcal{N}\left(\mathbf{z}_t; \sqrt{\bar{\alpha}_t}\,\mathbf{z}_0, (1 - \bar{\alpha}_t)\mathbf{I}\right)$$

where $\{\alpha_t\}_{t=1}^T$ is a predefined noise schedule, $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$, and $\mathbf{z}_t$ denotes the noisy latent at timestep t. In practice, this can be written as:

$$\mathbf{z}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{z}_0 + \sqrt{1 - \bar{\alpha}_t}\,\boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, I)$$

The goal of the model is to learn the reverse process, which reconstructs $\mathbf{z}_0$ from noisy samples $\mathbf{z}_t$. This is achieved by training a neural network ϵ_θ to predict the noise $\boldsymbol{\epsilon}$ added at each timestep.

To condition the model on the diffusion timestep, we encode t using a sinusoidal embedding:

$$\mathbf{s}_t = \text{SinusoidalEmbedding}(t)$$

The noisy latent $\mathbf{z}_t$ is first projected into a higher-dimensional feature space, and the timestep embedding is added to this representation:

$$\mathbf{h} = \text{Proj}(\mathbf{z}_t) + \mathbf{s}_t$$

The resulting representation is processed by a multilayer perceptron, which outputs an estimate of the noise:

$$\hat{\boldsymbol{\epsilon}} = \epsilon_\theta(\mathbf{z}_t, t)$$

Following [6], we adopt the ϵ-prediction parameterization and train the model using the objective:

$$\mathcal{L}_{\text{diff}} = \mathbb{E}_{t,\mathbf{z}_0,\boldsymbol{\epsilon}} \left[\|\boldsymbol{\epsilon} - \epsilon_\theta(\mathbf{z}_t, t)\|_2^2 \right]$$

During generation, the model starts from pure Gaussian noise and iteratively applies the learned reverse process to produce a clean latent $\mathbf{z}_0$, which is then decoded into a time series by the VAE decoder.

2.3 Training

The proposed framework is trained in two successive stages, as summarized in Algorithm 1.

In the first stage, a variational autoencoder is trained to learn a compact and structured latent representation of the time series. The encoder maps an input $\mathbf{x}$ to a latent distribution $q_\phi(\mathbf{z}|\mathbf{x})$, from which a latent sample $\mathbf{z}_0$ is obtained using the reparameterization trick. The decoder then reconstructs the signal, and the model is optimized using the reconstruction and KL-divergence losses described in the previous section.

Once the autoencoder has converged, its parameters are frozen. In the second stage, a latent diffusion model is trained on the encoded latent variables. Given a latent sample $\mathbf{z}_0$, a timestep t is drawn uniformly, and Gaussian noise is added according to the forward diffusion process:

$$\mathbf{z}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{z}_0 + \sqrt{1 - \bar{\alpha}_t}\,\boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, I)$$

The denoising network ϵ_θ is then trained to predict the noise $\boldsymbol{\epsilon}$ from $(\mathbf{z}_t, t)$ using a mean squared error objective.

2.4 Sampling

Once trained, the model can generate new time series by sampling from the learned latent distribution, as described in Algorithm 2.

The process starts by sampling a latent variable $\mathbf{z}_T \sim \mathcal{N}(0, I)$ corresponding to the final step of the diffusion process. The model then iteratively applies the learned reverse denoising dynamics to progressively remove noise and recover a clean latent representation $\mathbf{z}_0$. Finally, this latent code is decoded into a time series using the VAE decoder.

3 Evaluation Metrics

We evaluate the quality of the generated time series using both qualitative and quantitative metrics. Visual inspection of original data, autoencoder reconstructions, and generated samples provides an initial qualitative assessment of fidelity and diversity.

3.1 Qualitative Metrics

We employ t-distributed Stochastic Neighbor Embedding (t-SNE) [15] to project both real and generated time series into a two-dimensional space. This visualization allows us to assess whether the generated samples capture the local structure of the original distribution, with greater overlap indicating higher similarity.

We also use Kernel Density Estimation (KDE) to compare the marginal distributions of values between real and generated data. By constructing smooth density estimates using Gaussian kernels, KDE reveals whether the generated samples reproduce the value distribution of the original data, independently of temporal ordering.

3.2 Quantitative Metrics

Context-FID Score. We compute a Context-Fréchet Inception Distance (Context-FID) by embedding each time series into a feature space composed of basic statistics (mean, standard deviation, skewness, kurtosis), range characteristics (minimum, maximum, range), temporal dependencies (autocorrelation at lags 1, 2, and 3), and frequency information (average magnitudes of the three dominant FFT components). The resulting feature distributions for real and generated data are modeled as multivariate Gaussians, and their Fréchet distance is computed as

$$\text{FID} = \|\mu_r - \mu_g\|^2 + \text{Tr}\left(\Sigma_r + \Sigma_g - 2(\Sigma_r\Sigma_g)^{1/2}\right).$$

Here, (μ_r, Σ_r) and (μ_g, Σ_g) denote the mean and covariance of real and generated features, respectively. Lower values indicate closer alignment between the two distributions.

Correlational Score. To evaluate whether cross-variable dependencies are preserved, we compare the correlation matrices of real and generated data. The correlational score is defined as $\mathbb{E}\left[\|\mathbf{C}_r - \mathbf{C}_g\|_1\right]$, where $\mathbf{C}_r$ and $\mathbf{C}_g$ denote the correlation matrices of real and generated samples. Lower values indicate better preservation of multivariate relationships.

Discriminative Score. Following [16], we train a bidirectional LSTM classifier to distinguish between real and generated time series. We report the excess accuracy over random guessing, defined as $\max(0, \text{Accuracy} - 0.5)$. A value close to zero indicates that the classifier cannot reliably distinguish between real and generated samples, implying high generative fidelity.

Predictive Score. To assess the usefulness of synthetic data for downstream tasks, we adopt the Train on Synthetic, Test on Real (TSTR) protocol [16]. A two-layer LSTM is trained on synthetic sequences to predict future time steps and evaluated on real data using the mean squared error $\frac{1}{N}\sum_{i=1}^{N}\|\hat{\mathbf{x}}^{(i)}_{T-\Delta+1:T} - \mathbf{x}^{(i)}_{T-\Delta+1:T}\|^2$. Lower values indicate that the generated data captures temporal dynamics that generalize to real sequences.

4 Experiments and Dataset

We were not able to reproduce the performance reported in the original paper. We therefore propose to analyze two variants of the proposed method, introducing slight modifications in order to obtain more stable and satisfactory results.

Experiments are conducted on a synthetic dataset of multivariate sine time series with five features. Each feature follows a distinct distribution, with frequencies sampled from a Gaussian law.

5 Implementations

5.1 Hyperparameters

The TimeLDM framework involves several hyperparameters controlling both the VAE and the latent diffusion model. Following [11], the reconstruction loss combines three terms with weights:

- λ_1: weight for the L2 loss, capturing global amplitude differences

- λ_2: weight for the L1 loss, improving robustness to outliers
- λ_3: weight for the FFT loss, preserving periodic structures

The VAE uses adaptive β scheduling with:

- $\beta_{\max} = 10^{-2}$
- $\beta_{\min} = 10^{-5}$
- $\lambda = 0.7$ (decay factor)
- $S = 5$ (patience)

The latent diffusion model uses the noise schedule $\sigma(t) = t$ over horizon T. These values are chosen empirically to balance reconstruction fidelity and generation diversity.

5.2 Implementation 1

The first implementation follows the original TimeLDM architecture [11]. The model consists of a transformer-based VAE and a diffusion model operating on the latent space.

The VAE uses two transformer encoders to estimate μ and σ, and a decoder with self-attention and cross-attention layers. The diffusion model reshapes the latent representation, applies a linear projection, adds sinusoidal time embeddings, and processes the result with a multilayer perceptron. The forward process follows $\mathbf{z}_t = \mathbf{z}_0 + t \cdot \epsilon$ with $\sigma(t) = t$.

We observe that the reconstruction loss weights strongly affect performance. With a small FFT weight ($\lambda_3 = 0.01$), the autoencoder reconstructs the input accurately, but the diffusion model fails to learn the latent distribution. With a large FFT weight ($\lambda_3 = 1$), the FFT term dominates, and the decoder prioritizes frequency content at the expense of temporal structure.

We find that $\lambda_3 = 0.2$ provides the best trade-off between reconstruction fidelity and generation quality. We therefore use $\lambda_1 = \lambda_2 = 1$, $\lambda_3 = 0.2$, and $T = 1$.

We also consider an improved version of Implementation 1, where modifications to the diffusion process and architecture lead to better empirical results (see Table 1).

5.3 Implementation 2

The second implementation introduces several modifications to improve stability and representation quality.

Spectral normalization [10] is applied to constrain the Lipschitz constant. Batch normalization [7] and layer normalization [1] improve gradient flow. GELU [4] replaces ReLU, and the latent dimension is increased from 16 to 32.

KL warmup [12] is introduced, increasing the KL weight from 0 to 1 over the first 10 epochs. Residual connections [3] and dropout [14] are added. The optimizer is changed to AdamW [9], and Xavier initialization [2] is used.

The diffusion model uses a hidden dimension of 512 and concatenates time embeddings instead of adding them. The noise schedule is extended to $T = 500$, providing finer diffusion steps. The reverse process is approximated for computational efficiency.

5.4 Results Analysis

We evaluate both implementations in terms of temporal structure, distribution alignment, and frequency preservation. Figures 1, 2, and 5 show comparisons between real data, reconstructions, and generated samples.

Implementation 1 with $\lambda_3 = 1.0$ exhibits latent collapse, producing nearly constant reconstructions and low-diversity samples, although frequency components are preserved.

Reducing λ_3 to 0.2 improves both reconstruction and generation quality. The improved version further enhances temporal coherence while preserving frequency structure.

Implementation 2 produces more diverse samples but introduces noise, resulting in reduced temporal consistency.

5.5 Quantitative Evaluation

Table 1 presents the quantitative results. The discriminative score is 0.5 for all configurations, indicating that a classifier can perfectly distinguish between real and generated samples. This is consistent with the qualitative differences observed in Figures 1, 2, and 5.

The improved Implementation 1 achieves the best Context-FID score (2.47), showing strong statistical alignment. However, it exhibits higher predictive error. Implementation 1 with $\lambda_3 = 1.0$ achieves the lowest predictive score but suffers from poor diversity.

Table 1: Quantitative comparison of all implementations. Lower scores are better for all metrics.

Metric	**Impl. 1** ($\lambda_3 = 1.0$)	**Impl. 1** ($\lambda_3 = 0.2$)	**Improved Impl. 1**	**Impl. 2**
Context-FID Score	16.0725	12.2660	**2.4729**	8.7116
Correlational Score	0.0991	**0.0603**	0.1129	0.1070
Discriminative Score	0.5000	0.5000	0.5000	0.5000
Predictive Score	**0.5059**	0.7558	0.7193	0.5237

5.6 Discussion

The results highlight several key observations. The parameter λ_3 controls the trade-off between reconstruction fidelity and latent space structure. Large values lead to latent collapse, while smaller values allow more expressive representations.

Architectural improvements increase sample diversity but can introduce temporal noise, indicating that regularization must be carefully tuned.

Overall, the improved Implementation 1 provides the best balance between statistical fidelity and generative diversity. However, all configurations remain distinguishable from real data, indicating that further improvements are required.

6 xAI analysis

Explainability of deep generative models remains a challenging problem, particularly in the context of diffusion-based architectures. In classical settings such as image classification, interpretability methods like saliency maps or gradient-based attribution provide meaningful insights into the regions of the input that drive model predictions. Similarly, global sensitivity analysis techniques such as Sobol' indices allow one to quantify the contribution of input features to the variance of the model output.

However, these approaches are not directly applicable to diffusion models. In our setting, the model does not map a fixed input to an output, but instead generates samples by iteratively denoising an initial random latent variable. As a result, there is no single deterministic input-output relationship to analyze. Gradient-based methods become difficult to interpret, as gradients propagate through a multi-step stochastic process, and do not correspond to localized or human-interpretable features in the generated signal.

Moreover, our framework operates in a learned latent space obtained through a variational autoencoder, further complicating interpretation. Understanding the role of latent variables, the influence of noise at different timesteps, and the mechanisms by which temporal structures emerge requires adapted explainability tools.

To address these challenges, we explore three complementary approaches. First, we use Sobol' indices to perform a global sensitivity analysis of the latent variables and quantify their influence on the generated time series. Second, we introduce a notion of **series saliency**, aimed at identifying which parts of the temporal signal are most influenced by the generative process. Finally, we investigate counterfactual explanations, which provide insight into how modifications of latent variables or intermediate states affect the generated outputs.

6.1 Definition of the target features

In this explainability analysis, we assess the ability of the model to generate time series exhibiting specific statistical and spectral properties. To this end, we define a set of scalar target features computed from the generated signals.

In particular, we focus on three key characteristics of a selected signal dimension:

- **Mean value**: captures the global amplitude and bias of the signal.
- **Dominant frequency**: defined as the location of the main peak in the Fourier transform, characterizing the periodic structure of the signal.
- **Signal energy**: defined as the squared ℓ_2 norm of the signal, reflecting its overall intensity.

6.2 Sobol' indices

Sobol' indices are a class of global sensitivity analysis methods that quantify how the variance of a model output can be attributed to its input variables. Let $Y = f(X_1, \ldots, X_d)$ be a scalar output depending on input variables $X_1, \ldots, X_d$. The total variance of the output can be decomposed as

$$\mathrm{Var}(Y) = \sum_{i=1}^{d} V_i + \sum_{i<j} V_{ij} + \cdots + V_{1\ldots d},$$

where $V_i = \mathrm{Var}(\mathbb{E}[Y \mid X_i])$ represents the contribution of variable X_i, and higher-order terms capture interactions between variables.

The **first-order Sobol' index** is defined as

$$S_i = \frac{\mathrm{Var}(\mathbb{E}[Y \mid X_i])}{\mathrm{Var}(Y)},$$

and measures the direct contribution of X_i to the output variance. The **total Sobol' index** is given by

$$S_{T_i} = 1 - \frac{\mathrm{Var}(\mathbb{E}[Y \mid X_{\sim i}])}{\mathrm{Var}(Y)},$$

and captures the total contribution of X_i, including all interaction effects.

In our setting, Sobol' indices are applied to the latent variables of the diffusion model. Each latent channel (across all time steps) is treated as an input variable group, and the output Y corresponds to a scalar feature of the generated time series (such as mean value, dominant frequency, or signal energy).

However, applying Sobol' analysis in this context presents several challenges. First, the inputs to the generative process are Gaussian noise variables, which are not directly interpretable. Here the latent variables do not have an immediate semantic interpretation.

Second, the generative process itself is stochastic and iterative: the final output is obtained through a multi-step denoising procedure starting from random noise. As a result, the model does not define a simple deterministic mapping $Y = f(X)$, but rather a stochastic transformation.

Consequently, the Sobol' indices do not yield spatially or temporally interpretable "importance maps" as in image-based models, even when averaging over multiple Sobol simulations. Instead, they provide a coarse

measure of how different latent channels influence global properties of the generated signal. Moreover, due to the stochastic nature of the generation process, the estimated indices may vary across runs, making their interpretation less stable.

Figures 4a and 4b highlight the instability of Sobol' sensitivity estimates in the context of our time series generation model. To mitigate the effect of randomness in the latent initialization, we generate 40 independent samples, compute the Sobol' indices for each run, and average the results.

Despite this averaging, we observe significant variability in the estimated sensitivities across runs. Moreover, the total Sobol' indices remain close to zero for most latent channels, indicating a weak global sensitivity of the output feature with respect to individual latent dimensions.

This suggests that the considered statistical feature of the generated signal does not depend strongly on any single latent channel. Instead, the generation process appears to rely on distributed and interaction-driven mechanisms, where multiple latent variables jointly influence the output. Consequently, there is no single dominant latent factor that explains the observed behavior, highlighting the entangled nature of the latent representation.

6.3 Series perturbation and channel ablation

We evaluate the contribution of each latent channel to the generation of time series. To this end, we introduce controlled perturbations by applying incremental and decremental variations to specific latent channels. This allows us to analyze how small changes in the latent representation affect the generated outputs, in particular whether they lead to mode collapse or shifts in the generated patterns.

In addition, we perform channel ablation by setting individual latent channels to zero. This enables us to assess the robustness of the generation process and to identify channels that are critical for preserving the structure and diversity of the generated time series.

Figures 5a and 5b highlight several interesting behaviors. Increasing perturbations applied to certain latent channels lead to a progressive collapse of the generated output dimensions, indicating a strong sensitivity of the model to these directions in latent space.

In contrast, complete ablation of individual channels (i.e., setting them to zero) still results in relatively robust time series generation. This suggests that the model exhibits a degree of redundancy in its latent representation and does not rely exclusively on any single channel to produce coherent outputs.

6.4 Counterfactual explanations

We analyze the generative model by optimizing the latent representation to enforce structured transformations in the generated time series. Given an initial latent variable $\mathbf{z}_0$, we seek a modified latent $\mathbf{z}_{cf}$ such that specific output channels are exchanged while keeping the latent close to its original value:

$$\mathbf{z}_{cf} = \arg\min_{\mathbf{z}} \ \mathcal{L}_{\text{swap}}(f(\mathbf{z})) + \lambda_{\text{prox}} \|\mathbf{z} - \mathbf{z}_0\|_2^2,$$

where $f(\mathbf{z})$ denotes the generative process.

For instance, swapping two output channels i and j is enforced through the loss:

$$\mathcal{L}_{\text{swap}} = \|x_{cf}^{(i)} - x_{\text{orig}}^{(j)}\|_2^2 + \|x_{cf}^{(j)} - x_{\text{orig}}^{(i)}\|_2^2,$$

where $x_{cf} = f(\mathbf{z}_{cf})$ and $x_{\text{orig}} = f(\mathbf{z}_0)$.

This formulation allows us to probe whether channel-specific patterns can be reassigned through latent perturbations. Ideally, in a disentangled representation, such a transformation would only affect the targeted channels while leaving the others unchanged. Our experiments on figure 6 show that enforcing channel swapping partially preserves the signals of the other channels, suggesting that the latent space is only weakly disentangled and that information is distributed across multiple latent dimensions.

This observation is consistent with prior work on variational autoencoders and representation learning, where disentanglement depends on the strength of the latent regularization. In particular, the β-VAE framework [5] shows that increasing the weight of the Kullback–Leibler divergence promotes more factorized representations, at the cost of reconstruction fidelity.

7 Use of Artificial Intelligence

7.1 Theoretical Clarification

The TimeLDM paper [11] presents a complex architecture combining transformer-based VAEs with score-based diffusion models. When encountering ambiguous formulations, particularly regarding the interaction between self-attention and cross-attention mechanisms in the decoder (Figure 2d) and the precise implementation of the reverse SDE, we consulted ChatGPT to obtain alternative explanations. These clarifications helped us resolve implementation ambiguities but were always verified against the original paper and foundational literature [6, 13, 8]. We treated AI-generated explanations as supplementary interpretations rather than authoritative sources, cross-referencing with academic literature to ensure correctness.

7.2 Technical Implementation and Debugging

During implementation in Google Colab, we encountered several non-trivial bugs, including shape mismatches in attention tensors, gradient instability during VAE training, and difficulties in reproducing the reverse diffusion sampling procedure. We used Gemini to assist in debugging by:

- Identifying tensor dimension inconsistencies in transformer layers
- Suggesting gradient clipping thresholds based on empirical best practices
- Recommending specific PyTorch functions for spectral normalization implementation

However, all suggested fixes were critically evaluated: we tested multiple alternatives, compared their effects on training dynamics, and only incorporated modifications after understanding their theoretical justification. For instance, when Gemini proposed modifying the attention dropout rate, we systematically varied this hyperparameter and observed its effect on reconstruction fidelity before finalizing the architecture.

7.3 Methodological Exploration

To enhance model stability and output diversity, we adopted an iterative experimental approach guided by AI suggestions. Rather than blindly implementing recommendations, we:

1. Generated a set of potential architectural modifications from the AI (e.g., spectral normalization, residual connections, KL warmup)
2. Researched the theoretical motivation for each technique in the literature [10, 3, 12]
3. Implemented them incrementally, isolating their individual contributions
4. Retained only modifications that provided measurable improvement without degrading other metrics

This methodology ensures that AI served as an idea generator rather than an authority, with final architectural decisions grounded in empirical validation.

7.4 Editorial Assistance

For the manuscript, we composed the initial draft independently, then used AI to identify grammatical errors and awkward phrasings. All substantive scientific content including experimental results, methodology descriptions, and interpretations remains entirely our own. We carefully reviewed every AI suggestion to ensure it preserved technical accuracy and did not introduce scientific errors or exaggerations.

8 Conclusion and Future Work

8.1 Conclusion

In this work, we investigated latent diffusion models for time series generation through the TimeLDM framework, combining a transformer-based variational autoencoder with a diffusion process in latent space. While successful in image generation, our study highlights important limitations when applied to time series data.

From an empirical standpoint, we were not able to reproduce the performance reported in the original paper. Despite exploring multiple implementations and improvements, all configurations remain clearly distinguishable from real data. In particular, the discriminative score consistently indicates that generated samples can be separated from real ones, suggesting that the model fails to capture the true data distribution.

Our explainability analysis reveals structural issues in the latent representation. Counterfactual experiments show that small structured modifications in the output space (e.g., swapping two channels) require large and non-local changes in the latent variables, often affecting unrelated components. This indicates that the latent space is not stable and only weakly disentangled, with information distributed across dimensions.

Perturbation and ablation studies further highlight a significant redundancy in the latent space. While some perturbations lead to output collapse, removing individual latent channels often has limited impact, showing that the model does not efficiently use its latent capacity.

The Sobol' sensitivity analysis supports these findings: estimated indices are low and unstable, with most latent channels contributing negligibly to output variance. This suggests that generation relies on complex interactions rather than structured or interpretable factors.

Taken together, these results indicate that latent diffusion models are not yet well suited for time series, which require stronger handling of temporal dependencies and more stable representations.

8.2 Future Work

Several directions could address these limitations. First, enforcing a more structured latent space through disentanglement objectives, sparsity constraints, or information bottlenecks could reduce redundancy and improve interpretability.

Second, incorporating time series-specific inductive biases, such as temporal or causal structure, may improve the modeling of underlying dynamics. Hybrid approaches combining diffusion with sequence models (e.g., RNNs or transformers) could provide a more suitable framework.

Third, the diffusion process itself could be adapted to sequential data, as current noise schedules and objectives do not explicitly capture temporal dependencies.

Additionally, integrating sensitivity analysis into training (e.g., Sobol-based regularization) could help balance the contribution of latent variables and reduce instability.

Finally, alternative or hybrid generative approaches, such as autoregressive or flow-based models, may offer stronger inductive biases for time series generation.

In summary, while promising, latent diffusion models still require significant methodological improvements to become reliable tools for time series generation.

References

[1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[2] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 249–256, 2010.

[3] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

[4] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*, 2016.

[5] Irina Higgins, Loic Matthey, Arka Pal, Christopher Burgess, Xavier Glorot, Matthew Botvinick, Shakir Mohamed, and Alexander Lerchner. beta-vae: Learning basic visual concepts with a constrained variational framework. In *International Conference on Learning Representations (ICLR)*, 2017.

[6] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33:6840–6851, 2020.

[7] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International Conference on Machine Learning (ICML)*, pages 448–456, 2015.

[8] Tero Karras, Miika Aittala, Timo Aila, and Samuli Laine. Elucidating the design space of diffusion-based generative models. *Advances in Neural Information Processing Systems*, 35:26565–26577, 2022.

[9] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

[10] Takeru Miyato, Toshiki Kataoka, Masanori Koyama, and Yuichi Yoshida. Spectral normalization for generative adversarial networks. In *International Conference on Learning Representations (ICLR)*, 2018.

[11] Jian Qian, Bingyu Xie, Biao Wan, Minhao Li, Miao Sun, and Patrick Yin Chiang. Timeldm: Latent diffusion model for unconditional time series generation. *arXiv preprint arXiv:2407.04211*, 2024.

[12] Casper Kaae Sønderby, Tapani Raiko, Lars Maaløe, Søren Kaae Sønderby, and Ole Winther. Ladder variational autoencoders. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 29, 2016.

[13] Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. *arXiv preprint arXiv:2011.13456*, 2020.

[14] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research (JMLR)*, 15(1):1929–1958, 2014.

[15] Laurens Van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of Machine Learning Research*, 9(11):2579–2605, 2008.

[16] Jinsung Yoon, Daniel Jarrett, and Mihaela Van der Schaar. Time-series generative adversarial networks. *Advances in Neural Information Processing Systems*, 32, 2019.

9 Appendix

9.1 The TimeLDM algorithm

The **TimeLDM training** algorithm is highlighted below.

Algorithm 1 Training Algorithm of TimeLDM

Require: Time series dataset $\mathbf{S} = \{\mathbf{x}_i\}_{i=1}^{N}$
Ensure: Encoder $\mathcal{E}_\phi$, Decoder $\mathcal{D}_\xi$, denoiser ϵ_θ

```
Initialize E_φ, D_ξ
while autoencoder not converged do
    Sample x ~ S
    (μ, σ) ← E_φ(x)
    ε ~ N(0, I)
    z_0 ← μ + σ ⊙ ε
    x̂ ← D_ξ(z_0)
    L_VAE ← L_recon + β L_KL
    Update φ, ξ
end while
Freeze E_φ, D_ξ
Initialize ε_θ
while diffusion model not converged do
    Sample x ~ S
    z_0 ~ q_φ(z|x)
    t ~ U({1, ..., T})
    ε ~ N(0, I)
    z_t ← √ᾱ_t z_0 + √(1 − ᾱ_t) ε
    L_diff ← ‖ε_θ(z_t, t) − ε‖_2^2
    Update θ
end while
return E_φ, D_ξ, ε_θ
```

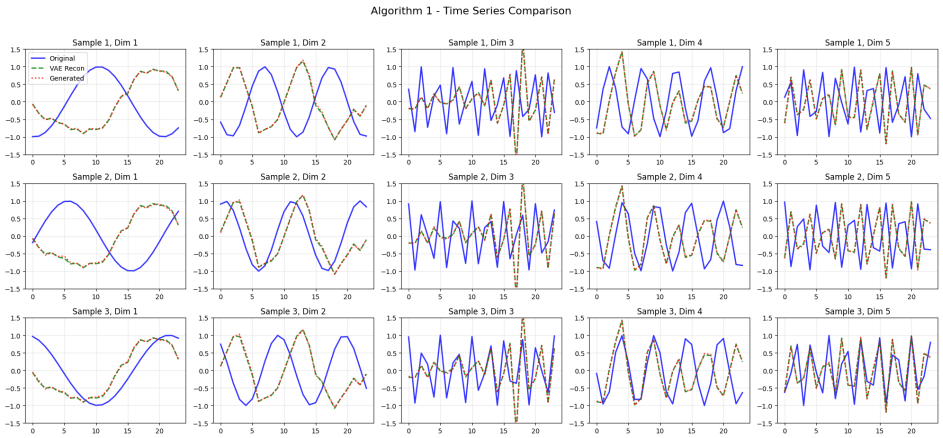
The **TimeLDM sampling** algorithm is highlighted below.

Algorithm 2 Sampling Algorithm of TimeLDM

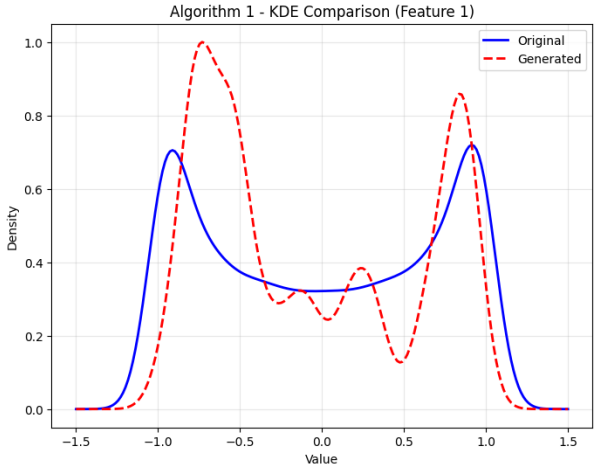
Require: Decoder $\mathcal{D}_\xi$, denoiser ϵ_θ
Ensure: Generated time series $\hat{\mathbf{x}}$

```
z_T ~ N(0, I)
for t = T, ..., 1 do
    ε̂ ← ε_θ(z_t, t)
    z_{t−1} ← reverse step(z_t, ε̂, t)
end for
x̂ ← D_ξ(z_0)
return x̂
```

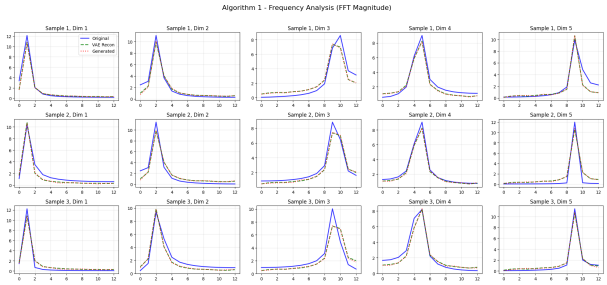
9.2 Different implementations results



(a) Time series comparison

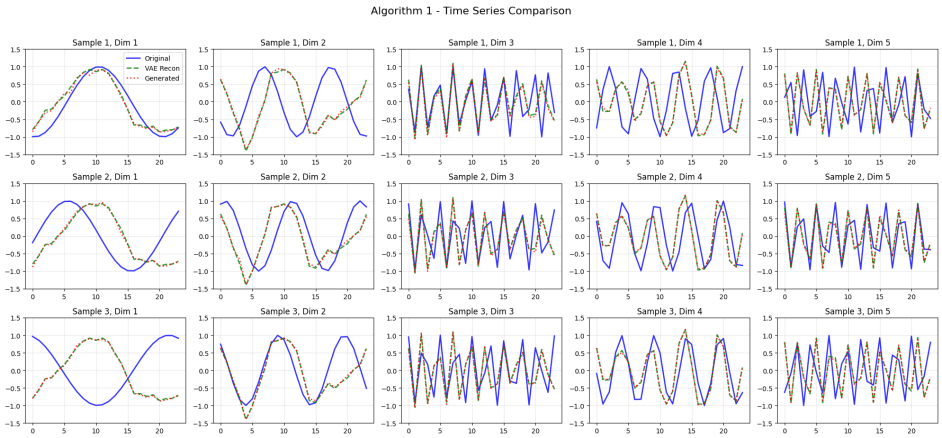


(b) Kernel density estimation

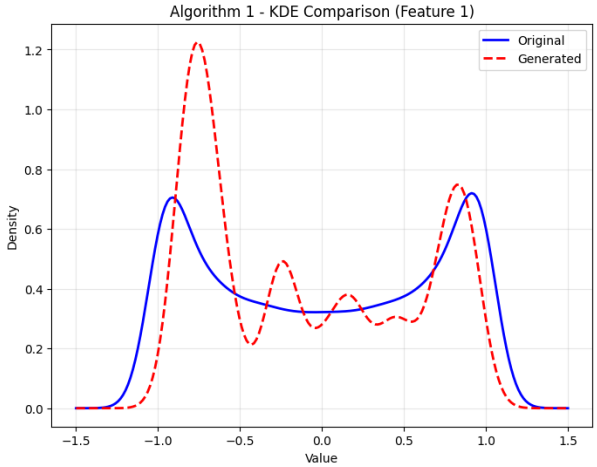


(c) Frequency spectrum

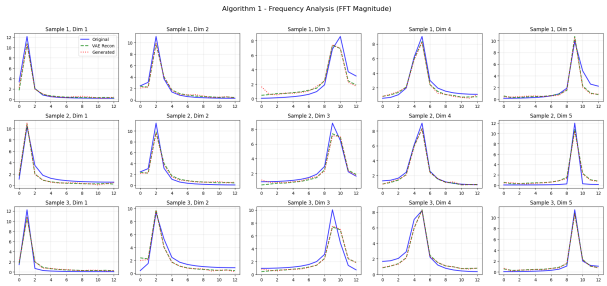
Figure 1: Implementation 1 with $\lambda_3 = 1.0$. The autoencoder produces nearly constant reconstructions regardless of input, indicating a collapse in latent representation. The generated samples closely mirror these collapsed reconstructions, resulting in poor diversity but accurate frequency preservation. The KDE shows good alignment, as the generated distribution matches the autoencoder output.



(a) Time series comparison

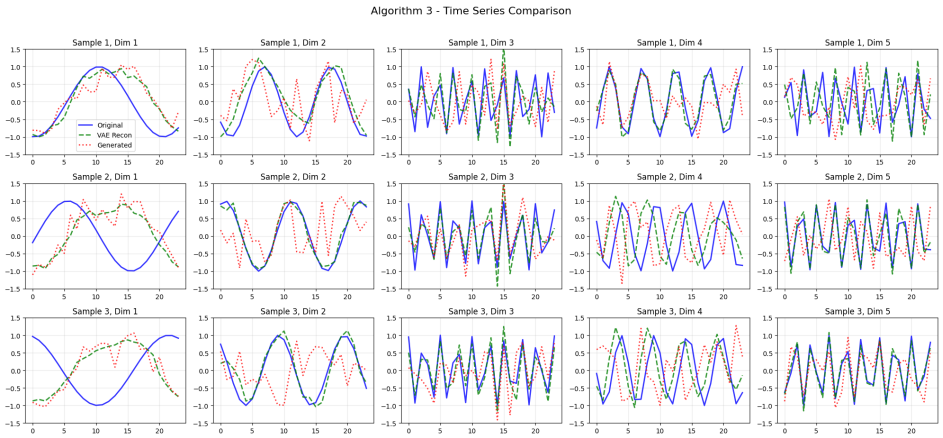


(b) Kernel density estimation

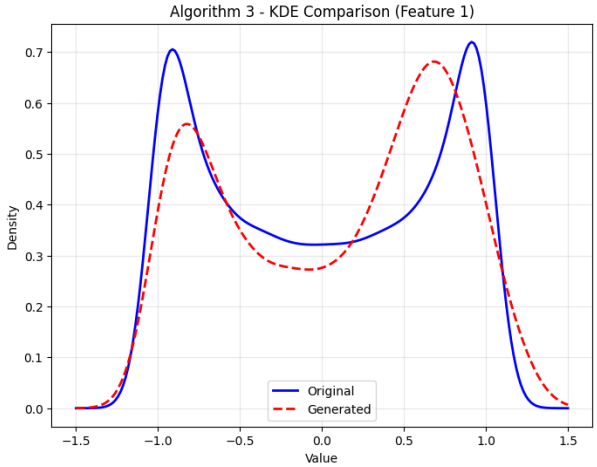


(c) Frequency spectrum

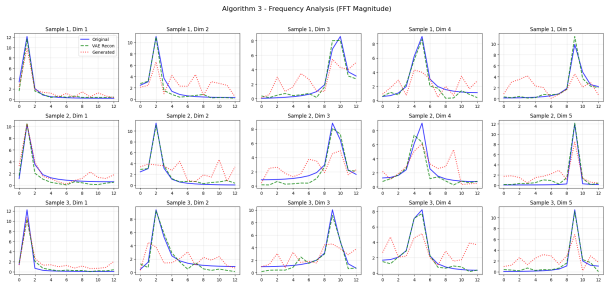
Figure 2: Improved Implementation 1. The autoencoder reconstructions exhibit greater fidelity to the original signals, and the generated samples show improved diversity while maintaining temporal coherence. The frequency spectrum is well preserved, indicating that the model captures periodic structures effectively.



(a) Time series comparison



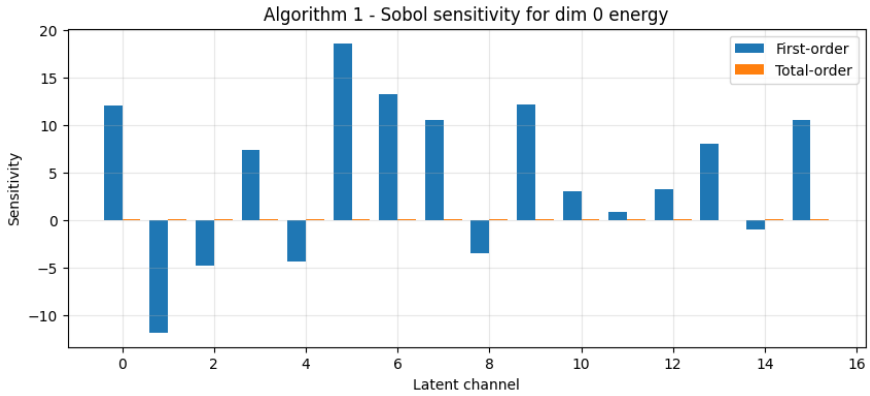
(b) Kernel density estimation



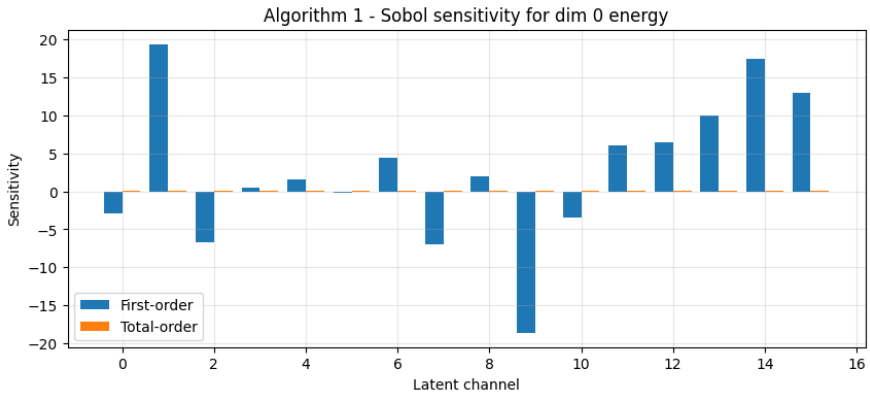
(c) Frequency spectrum

Figure 3: Implementation 2 with architectural enhancements. While the generated samples exhibit greater diversity, they also display increased noise and reduced temporal coherence compared to Implementation 1. The KDE shows reasonable alignment with the original distribution, and frequency components are partially preserved.

9.3 Highlighting Sobel "insensitivity"



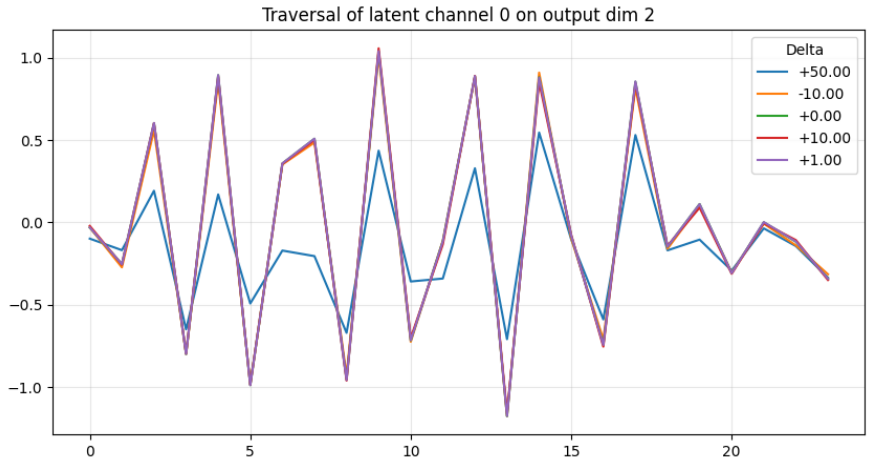
(a) First generation Sobol sensitivity



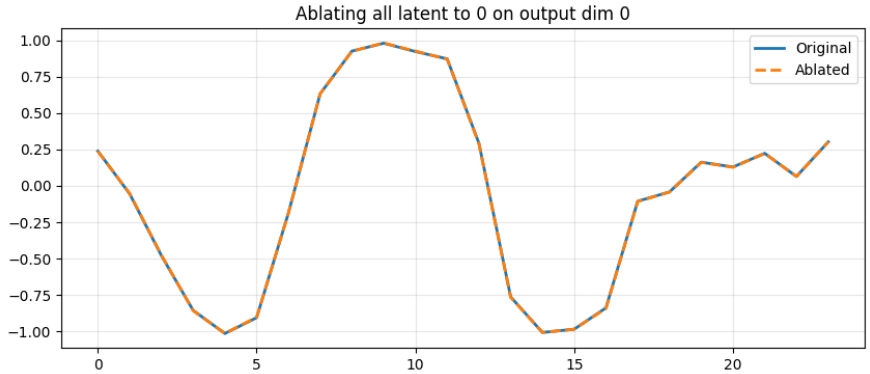
(b) Second generation Sobol sensitivity

Figure 4: Both figures highlight the Sobol' sensitivity indices for two different generation processes. Because of the stochastic nature of the denoising procedure, it is not possible to precisely determine which latent variables are responsible for the variance of the outputs.

9.4 Series perturbation and channel ablation



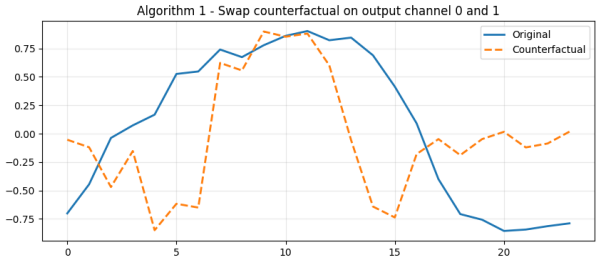
(a) Perturbing the first channel of the latent dimension



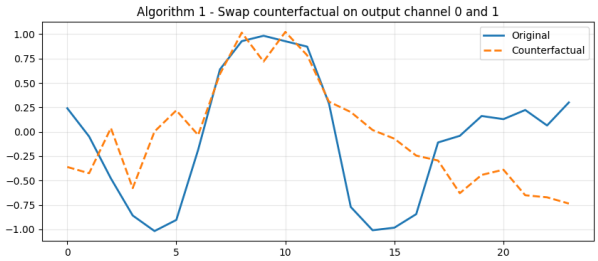
(b) Latent space ablation

Figure 5: The first figure illustrates the effect of latent perturbations on the model outputs. The second figure highlights an interesting property of the diffusion-based generation process. Even when the input latent variable is fixed to zero, the denoising and decoding steps still produce a realistic-looking time series. This behavior suggests that the model has learned a strong prior over the data distribution, enabling it to generate plausible signals even from degenerate or non-informative latent inputs.

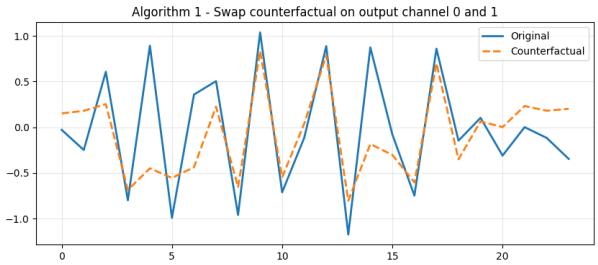
9.5 Counterfactual explanations



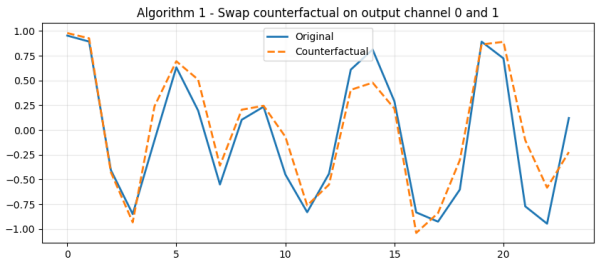
(a) Case 1



(b) Case 2



(c) Case 3



(d) Case 4

Figure 6: Counterfactual experiments obtained by optimizing the latent variable $\mathbf{z}$ to enforce a channel-swapping transformation between output channels 0 and 1. The results show that the desired swap can be partially achieved, while other channels remain relatively preserved. This behavior highlights the partially disentangled nature of the latent representation, where channel-specific patterns can be manipulated but remain coupled across latent dimensions.